UČNO GRADIVO

Vodnik po Engineering Investor

Kako razumeti, urejati in objavljati spletno stran

Astro 6 · React 19 · GitHub Pages

Verzija 1.0 · Maj 2026



Kazalo

Ta vodnik te po korakih nauči, kako je sestavljena spletna stran in kako jo varno spreminjaš. Ne potrebuješ predznanja programiranja, dovolj je radovednost.

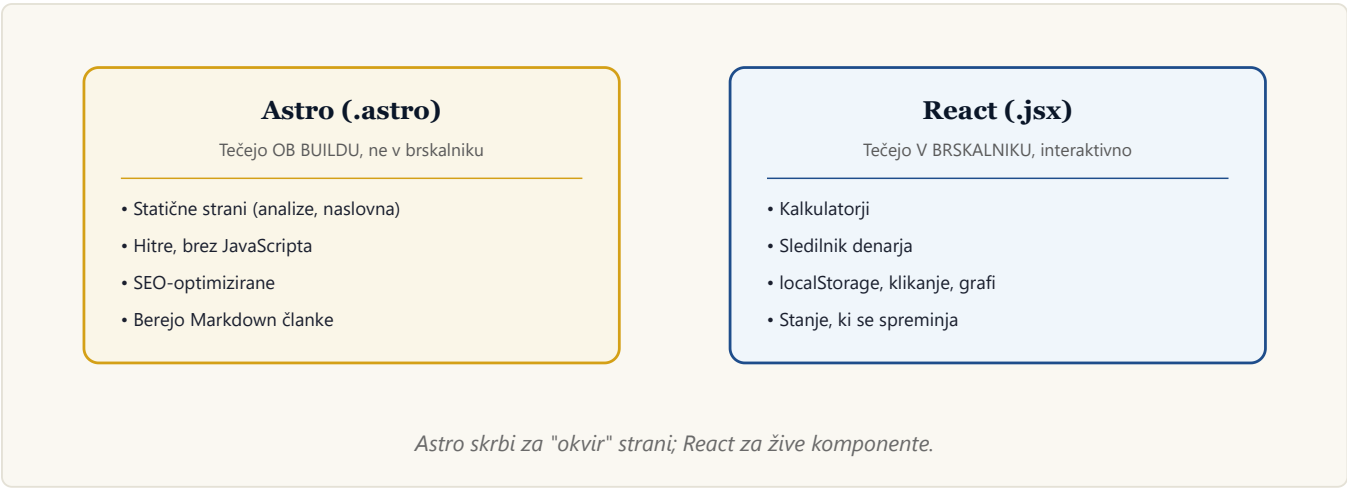
1. Veliki vzorec — kako stvar deluje
2. Mape — kaj je kje
3. Kje spreminjati kaj — 16 tipičnih nalog
4. Razvojni cikel — kako delaš spremembe
5. Git osnove — shranjevanje in objavljanje
6. Kako razmišljati pri programiranju
7. Dnevni pregled — posebnost
8. Tipičen scenarij — dodajam nov članek
9. Pogosta orodja, ki jih boš srečal
10. Naprej — kaj se naučiti
11. Hitri referenčni listek

1 Veliki vzorec — kako stvar deluje

Spletna stran je narejena z **Astro** (ogrodje), z **React** komponentami za interaktivne dele. Stran je **statična**: ko jo zgradimo, nastane mapa `dist/` z gotovimi HTML/CSS/JS datotekami. Te se hostajo na **GitHub Pages** brezplačno.



Astro vs React — razlika



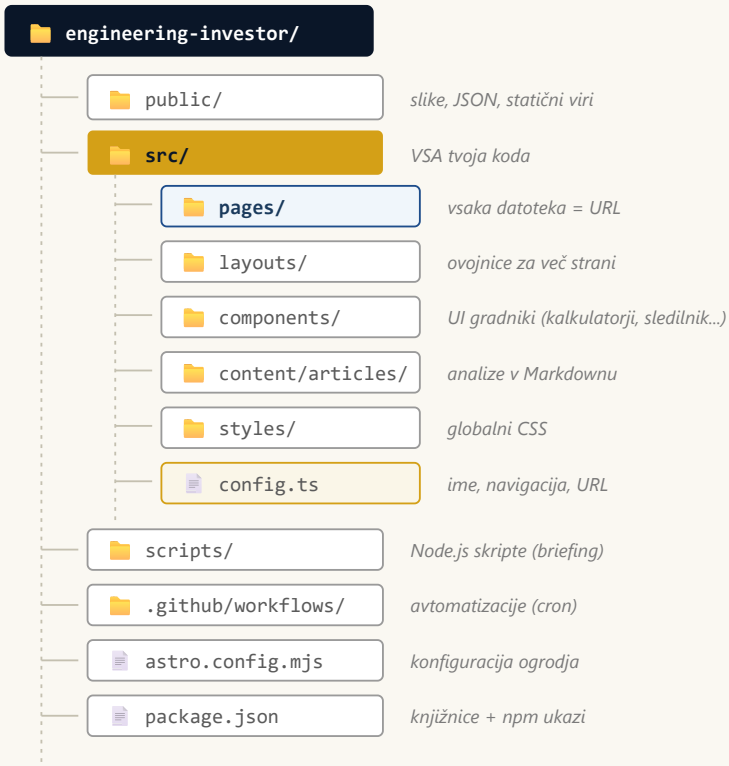
Astro datoteka lahko **vključi** React komponento takole:

```
<MoneyTracker client:only="react" />
```

`client:only="react"` pomeni: zaženi to komponento šele v brskalniku.

2 Mape — kaj je kje

Drevo projekta



Rumeno = najpomembnejše datoteke za začetnika.

2.1. `src/pages/` — strani

Vsaka datoteka v tej mapi postane URL na strani. To se imenuje *file-based routing*.

File-based routing

Datoteka		URL
`src/pages/index.astro`	→	`engineeringinvestor.si/`
`src/pages/o-meni.astro`	→	`engineeringinvestor.si/o-meni`
`src/pages/sledilnik.astro`	→	`engineeringinvestor.si/sledilnik`
`src/pages/analize/index.astro`	→	`engineeringinvestor.si/analize`
`src/pages/analize/[slug].astro`	→	`engineeringinvestor.si/analize/{slug}`

Vsaka stran običajno samo **uvozi** ovojnico in vsebino:

```
---
import BaseLayout from '../layouts/BaseLayout.astro';
import MoneyTracker from '../components/tracker/MoneyTracker.jsx';
---
<BaseLayout title="Sledilnik">
  <MoneyTracker client:only="react" />
</BaseLayout>
```

> **PRAVILO**
> Kadarkoli želiš novo URL pot na strani, dodaj datoteko v `src/pages/` .

2.2. `src/layouts/` — ovojnice

V `BaseLayout.astro` je vse, kar se ponavlja na vsaki strani: glava (`<head>`), navigacija, noga, meta tagi. Posamezne strani (`pages/`) samo "padejo noter" v to ovojnico.

Če bi rad spremenil **navigacijo** (zgornji meni), to NI v `BaseLayout` , ampak v `src/components/layout/` in seznam povezav je v `src/config.ts` .

2.3. `src/components/` — komponente

Tu so vsi gradniki UI, razdeljeni po smiselnih podkategorijah:

```
components/
├── articles/      # Komponente za prikaz analiz
├── calculators/   # 3 kalkulatorji (naložb, INR vs IBKR, neto vrednost)
├── home/          # Naslovna stran (hero, najnovejše analize)
├── layout/        # Glava in noga strani
├── news/          # Dnevni pregled (MarketBriefing.jsx)
├── premium/       # Premium komponente
├── tracker/       # Sledilnik denarja
└── ui/            # Mali generični gradniki (Badge, Disclaimer)
```

Komponenta je samostojen kos UI z lastno logiko. Princip: **iste stvari pišeš ENKRAT, uporabljaš večkrat.**

Primer: gumb "+" se uporablja v transakcijah, računih, ciljih. Zato imamo `Button` komponento v `tracker/ui.jsx` , ki jo vsi uvozijo:

```
import { Button } from '../ui.jsx';
<Button onClick={...}>+ Nov račun</Button>
```

2.4. `src/content/` — članki v Markdown

Analize (poglobljene pojasnjevalne objave) niso programirane, ampak **napisane v Markdownu** — to je preprost format brez HTML-ja.

```
content/articles/
├── trump-hormuz-blokada-2026.md
├── fed-obrestne-mere-2025.md
└── ...
```

Vsak `.md` ima na vrhu **frontmatter** (metapodatki) in pod njim besedilo:

```
---
title: 'Trump in Hormuz blokada'
date: 2026-04-15
category: 'Geopolitika'
excerpt: 'Kratek povzetek...'
---

# Glavni naslov

Tu pišeš članek.
```

`src/content.config.ts` določa **schema** — katera polja so obvezna v vsakem članku. Ko zaženeš build, Astro avtomatsko ustvari URL za vsakega: `/analize/trump-hormuz-blokada-2026`.

2.5. `public/` — statični viri

Vse v `public/` se servira **direktno** brez obdelave. Slike, fonti, podatki.

```
public/
├── data/briefing.json     # Tu se shrani rezultat dnevnega pregleda
├── og-image.png           # Slika za Twitter/Facebook share
└── favicon.svg
```

URL primer: datoteka `public/data/briefing.json` je dosegljiva na `engineeringinvestor.si/data/briefing.json`.

2.6. `scripts/` — Node.js skripte

Skripte, ki **niso** del spletne strani — tečejo na strežniku ali v GitHub Actions. `scripts/generate-briefing.mjs` se zažene vsak dan ob 06:30 UTC, pokliče NewsAPI in Gemini, in shrani rezultat v `public/data/briefing.json`.

2.7. `.github/workflows/` — avtomatizacije

GitHub Actions = brezplačni roboti, ki jih lahko nastaviš na sprožilce (cron, push, manualno). `daily-briefing.yml` ima dva dela:

1. **build** — vsak dan ob 06:30 UTC: zaženi skripto, shrani briefing, zgradi stran, naloži artefakt
2. **deploy** — naloži artefakt na GitHub Pages

2.8. `src/config.ts` — glavna konfiguracija

Tu je **ime strani, URL, navigacijski meni**. Če dodaš novo stran in želiš, da je v meniju, dodaj jo tu:

```
export const NAV_LINKS = [
  { label: 'Analize', href: '/analize' },
  { label: 'Kalkulatorji', href: '/kalkulatorji' },
  { label: 'Sledilnik', href: '/sledilnik' },
  ...
];
```

2.9. `astro.config.mjs` in `package.json`

V `astro.config.mjs` je base path (`/engineering-investor`) in vključeni vtičniki. **Tega običajno ne spreminjaš.**

V `package.json` so vse zunanje knjižnice in **npm skripte**:

```
"scripts": {
  "dev": "astro dev",       // razvojni strežnik
  "build": "astro build",   // gradnja produkcijske različice
  "preview": "astro preview" // pregled produkcijske različice
}
```

Zaženeš jih z `npm run dev`, `npm run build`, ipd.

3 Kje spreminjati kaj — 16 tipičnih nalog

Želim spremeniti...	Kje
Naslovno stran	`src/pages/index.astro` + komponente v `src/components/home/`
Navigacijski meni	`src/config.ts` (NAV_LINKS)
Glavo / nogo strani	`src/components/layout/`
Globalne barve / fonte	`src/styles/global.css`
Dodati novo analizo	Nov `.md` v `src/content/articles/`
Spremeniti obstoječo analizo	Pripadajoči `.md` v `src/content/articles/`
Dodati nov kalkulator	Nova `.jsx` v `src/components/calculators/` + nova stran v `src/pages/` + dodati v `src/pages/kalkulatorji.astro`
Dodati novo stran (npr. /blog)	Nova `.astro` v `src/pages/` + dodati v `src/config.ts`
Logiko sledilnika	`src/components/tracker/`
Skripto za dnevni pregled	`scripts/generate-briefing.mjs`
Cron urnik za dnevni pregled	`.github/workflows/daily-briefing.yml` (vrstica `cron:`)
Domeno / URL	`astro.config.mjs` (`site:`) + GitHub Pages nastavitve
Sliko za share na družbenih omrežjih	`public/og-image.png`
Favicon	`public/favicon.svg`
Sitemap	Avtomatsko (vtičnik `@astrojs/sitemap`)
Disclaimer / opozorila	`src/components/ui/Disclaimer.astro`

4 Razvojni cikel — kako delaš spremembe



Cikel ponavljaš za vsako spremembo. Po koraku 5 sledi avtomatika — počakaj ~2 minuti.

4.1. Lokalni razvoj — `npm run dev`

V terminalu (PowerShell/Git Bash), v mapi projekta:

```
cd "C:/Users/Jaž/OneDrive/Namizje/PROJEKTI/INVESTING SPLETNA STRAN/engineering-investor"
npm run dev
```

To zažene **lokalni strežnik** na `http://localhost:4321/engineering-investor`. Spremembe v kodi so vidne **takoj** (hot reload — ni potrebno osveževati ročno).

Ko si končal, v terminalu pritisni `Ctrl` + `C` da ustaviš strežnik.

4.2. Build — preverba pred objavo

```
npm run build
```

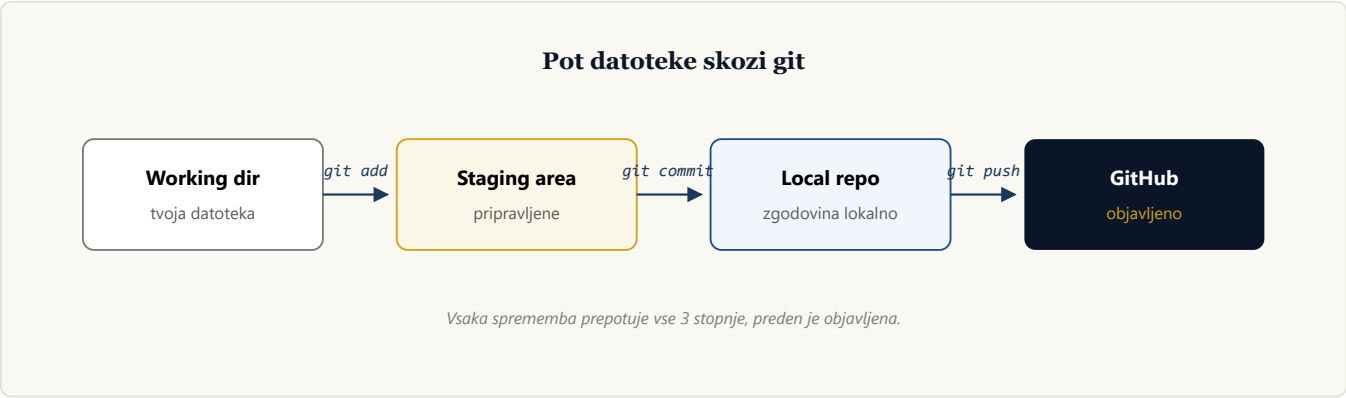
To naredi produkcijsko različico v mapo `dist/`. Če koda ni veljavna (sintaktične napake, manjkajoče komponente), bo build **padel** — to je dobro, ker vidi napako PRED objavo.

```
npm run preview
```

Servira `dist/` lokalno — vidiš, kako bo stran dejansko zgledala v produkciji.

5 Git osnove — shranjevanje in objavljanje

Git je sistem za vodenje verzij. Vsaka sprememba se shrani kot "commit" z opisom. **GitHub** je spletni servis, ki hostira tvoj git repozitorij.



5.1. Osnovni ukazi

```
git status            # Kaj je spremenjeno?
git diff              # Pokaži spremembe
git add <datoteka>    # Pripravi datoteko za commit
git add .             # Pripravi VSE spremembe (previdno!)
git commit -m "opis"  # Shrani spremembe v zgodovino
git push              # Naloži lokalne commite na GitHub
git pull              # Prenesi tuje spremembe iz GitHuba
git log               # Pokaži zgodovino commitov
```

5.2. Tipičen tok shranjevanja

```
# 1. Preveri stanje
git status

# 2. Izberi datoteke
git add src/components/tracker/views/Transactions.jsx

# 3. Naredi commit z opisom (kratek, opisen)
git commit -m "fix(tracker): clarify search placeholder"

# 4. Pošlji na GitHub
git push
```

PRAVILO DOBREGA COMMITA

Opis pove **kaj** se je spremenilo in **zakaj**, ne kako.

- Slabo: "spremembe"
- Slabo: "popravil sem datoteko Transactions.jsx z dodanim labelom"
- Dobro: "fix(tracker): clarify search placeholder to avoid confusion"

5.3. Konflikti pri push

Če pushaš in dobiš:

```
error: failed to push some refs
hint: Updates were rejected because the remote contains work that you do not have locally.
```

Pomeni, da je nekdo drug (ali GitHub Actions bot) pushal pred tabo. Reši takole:

```
git pull --rebase
git push
```

`--rebase` "potegne" tuje commite in tvoje postavi na vrh. Brez konfliktov gre samodejno.

5.4. Kaj NE delati

POZOR

- **Nikoli** `git push --force` na main brez razmisleka — prepiše tuje delo.
- **Nikoli** ne commitaj `.env` datotek z gesli ali API ključi.
- **Nikoli** ne commitaj mape `node_modules/` (je v `.gitignore`).
- **Ne** delaj velikih commitov z 50 spremembami — raje več manjših z jasnim opisom.

5.5. Skrivnosti / API ključi

API ključi (Gemini, NewsAPI) **niso** v kodi. So shranjeni kot **GitHub Secrets**: GitHub repo → Settings → Secrets and variables → Actions. V workflow yaml-u jih dostopaš z `${{ secrets.GEMINI_API_KEY }}`.

6 Kako razmišljati pri programiranju

Ko pišeš kodo, se sprašuj. Spodnja načela ne nadomeščajo prakse, ampak jo vodijo.

6.1. Razdeli problem na dele

Velika naloga ("zgradi sledilnik denarja") = **strašna**. Razdeljena na korake = **izvedljiva**:

1. Najprej shramba podatkov (kako shranim transakcijo?)
2. Potem prikaz seznama
3. Potem dodajanje
4. Potem urejanje
5. Potem grafi

Začni z **najmanjšim, najpreprostejšim delom, ki dela**, šele potem dodajaj.

6.2. DRY — Don't Repeat Yourself

Če pišeš isto kodo 3-krat, naredi **funkcijo** ali **komponento**. Primer: namesto da v vsaki view-u sledilnika pišeš `<button style={{ padding: '0.5rem'... }}>`, imamo `<Button>` komponento v `tracker/ui.jsx` — vse views jo uvozijo.

6.3. Imenovanja: kratka, jasna, povedna

```
slabo:    var x = data.filter(d => d.t > 0)
dobro:    const positiveTransactions = transactions.filter(t => t.amount > 0)
```

Tudi imena datotek: `Transactions.jsx`, ne `t.jsx` ali `transactions_v2_final.jsx`.

6.4. Beri obstoječo kodo, preden pišeš novo

Preden napišeš novo komponento, se vprašaj:

- *Mar že obstaja podobna?* (preglej `components/tracker/ui.jsx`, `components/ui/`)
- *Ali lahko obstoječo razširim?*

Tako se izogneš podvojeni kodi.

6.5. Single source of truth

Vsak podatek ima **eno mesto, kjer je shranjen**. Sledilnik: vse stanje je v `localStorage` pod ključem `ei-tracker-v1`, dostopano preko `loadState()` v `store.js`. Vsi viewi to stanje **berejo**, ne dvopisujejo lokalno.

6.6. Ko obstane, vprašaj "zakaj"

Če koda ne deluje:

1. **Preberi sporočilo o napaki natančno** — pogosto pove točno, kaj je narobe.
2. Vprašaj: *zakaj se to dogaja?*, ne *kako naj to skrijem?*
3. Dodaj `console.log(...)` za izpis vrednosti, da vidiš, kaj se zgodi.

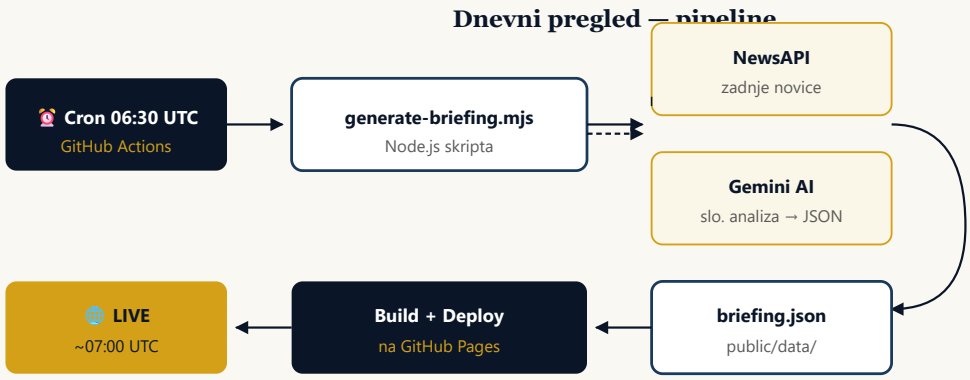
6.7. Ne piši kode "za bodočnost"

PRAVILO

Mlade razvijalce mami pisati abstraktno kodo "za bodoče potrebe". **To je past.** Piši najpreprostejšo kodo, ki reši **trenutni** problem. Refaktoriraš lahko, ko se potreba pojavi.

7 Dnevni pregled — posebnost

To je **edina avtomatizacija** na strani. Pretok prikazuje, kaj se zgodi vsak dan ob 06:30 UTC, brez tvojega posredovanja:



Komponenta MarketBriefing.jsx ob nalaganju strani fetcha briefing.json in prikaže.

Če želiš spremeniti **frekvenco**: `cron: '30 6 * * *'` v `.github/workflows/daily-briefing.yml`. Format: minute, ure, dan v mesecu, mesec, dan v tednu.

Če želiš spremeniti **prompt** za Gemini: `scripts/generate-briefing.mjs`, funkcija `buildUserPrompt`.

> **ROČNI SPROŽILEC**
>
> Lahko ga sprožiš tudi ročno: GitHub repo → Actions → Daily Market Briefing → Run workflow.

8 Tipičen scenarij — dodajam nov članek

Pretok od ideje do objave:

```
# 1. Naredi novo datoteko
src/content/articles/moja-nova-analiza.md
```

```
---
title: 'Moja nova analiza'
date: 2026-05-08
category: 'Trgi'
excerpt: 'Kratek povzetek za seznam.'
---

# Glavni naslov

Tu napišeš članek.
```

```
# 2. Lokalno preveri
npm run dev
# Odpri http://localhost:4321/engineering-investor/analize/moja-nova-analiza

# 3. Build preveri (naj ne bo napak)
npm run build

# 4. Shrani in objavi
git add src/content/articles/moja-nova-analiza.md
git commit -m "post: nova analiza o trgih"
git push
```

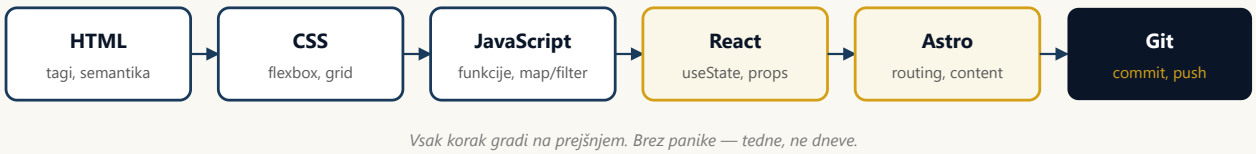
GitHub Actions bo zgradil in deployal v ~2 minutah.

9 Pogosta orodja, ki jih boš srečal

Orodje	Za kaj
VS Code	Urejevalnik kode (priporočam za to delo).
Terminal	PowerShell, Git Bash — za `npm`, `git` ukaze.
Brskalnik DevTools	`F12` — za inšpekcijo HTML/CSS in vidno napak v konzoli.
GitHub	Kjer je koda shranjena, kjer se sproži deploy.
node_modules/	Mapa s knjižnicami; `npm install` jo ustvari iz `package.json`. Ne commitaj je.

10 Naprej — kaj se naučiti

Vrstni red, ki priporočam:



Vsak korak gradi na prejšnjem. Brez panike — tedne, ne dneve.

1. **HTML osnove** — kaj so tagi, atributi, semantika (`<header>` , `<article>` , `<section>`).
2. **CSS osnove** — selektorji, flexbox, grid, custom properties (`--color-accent`).
3. **JavaScript osnove** — spremenljivke, funkcije, objekti, array metode (`map` , `filter` , `reduce`).
4. **React osnove** — komponente, props, useState, useEffect.
5. **Astro osnove** — frontmatter, file-based routing, integracije.
6. **Git osnove** — commit, push, pull, branch.

Brez znanja vseh detajlov razumeš **strukturo**, in to je dovolj, da lahko spreminjaš stran.

11 Hitri referenčni listek

Razvoj

```
npm run dev              # zaženi lokalno
npm run build            # zgradi produkcijsko
npm run preview          # pokaži produkcijsko lokalno
```

Git

```
git status               # kaj je spremenjeno
git add <datoteka>       # pripravi za commit
git commit -m "opis"     # shrani v zgodovino
git push                 # naloži na GitHub
git pull --rebase        # prenesi tuje + postavi svoje na vrh
```

Cron sprožilec ročno

```
GitHub repo → Actions → Daily Market Briefing → Run workflow
```

Pomembne lokacije v kodi

Navigacijski meni	`src/config.ts`
Globalni stili	`src/styles/global.css`
Sledilnik logika	`src/components/tracker/store.js`
Briefing skripta	`scripts/generate-briefing.mjs`
Cron urnik	`.github/workflows/daily-briefing.yml`

ZLATO PRAVILO

Če se kje obstaneš, vprašaj. **Bolje 5 minut vprašanja kot 5 ur ugibanja.**